Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Theory, Edge Cases & Code Examples

- ✓ The Optimizer's Fundamental Assumption
- ✓ Categories of Advanced Pitfalls
- ✓ Why This Is Dangerous
- ✓ Undefined Behavior Categories
- ✓ Detection Strategy

→ *Swipe to learn* →

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Theory

📌 1. The Optimizer's Fundamental Assumption

Core Principle:

The compiler optimizes based on what the C++ standard **allows**, not what the programmer **intends**.

Modern C++ compilers assume: **Your code contains ZERO undefined behavior (UB)**.

What This Enables:

Compiler Freedom	Example Transformation	Consequence
Remove entire code sections	Infinite loops with no side effects → deleted	"Impossible" code disappears
Reorder operations arbitrarily	Memory accesses reordered across statements	Threading bugs amplified
Assume "impossible" conditions	After `*p`, assume `p != nullptr`	Safety checks removed
Transform code unexpectedly	Overflow checks optimized away	Security vulnerabilities

---

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Theory

📌 2. Categories of Advanced Pitfalls

Category	Trap Examples	Debug Build	Release Build	Risk Level
Optimizer-Removed Code	Infinite loops, overflow checks, null checks after dereference	✅ Works	❌ Fails/Removed	⚠️⚠️⚠️ Critical
Move Semantics Confusion	Named rvalue references are lvalues, const rvalues copy	✅ Compiles	⚠️ Copies instead of moves	⚠️⚠️ Performance
Template Metaprogramming	SFINAE surprises, forwarding ref contexts	❌ Won't compile	❌ Won't compile	⚠️ Compile-time
Modern C++ (C++20)	constexpr vs consteval vs constinit	✅ May work	⚠️ Runtime cost	⚠️ Performance
Memory Model Subtleties	Alignment, padding, empty base optimization	✅ Works	⚠️ Slower/Larger	⚠️ Performance

---

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Theory

📌 3. Why This Is Dangerous

The Triple Threat:

1. **Passes Code Review**: Logic appears correct to human reviewers
2. **Works in Debug**: Optimizations disabled (`-O0`), code behaves as expected
3. **Fails in Production**: Optimizations enabled (`-O2` / `-O3`), code transformed unexpectedly

Autonomous Vehicle Context:

System	Potential Issue	Impact if UB Exploited
Sensor calibration loop	Infinite loop removed	Sensor never calibrates → blind vehicle
Speed limit check	Overflow check removed	Speed exceeds hardware limits → mechanical failure
Null pointer safety	Null check after dereference removed	Crash on invalid sensor data
Buffer management	Copy instead of move	100x memory bandwidth → real-time deadline missed

---

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Theory

📌 4. Undefined Behavior Categories

UB Type	Code Pattern	What Compiler Assumes	Result
Null dereference	`*p; if(p==nullptr)...`	If reached, p can't be null	Check removed
Signed overflow	`if (x+y < x)...`	Overflow never happens	Check always false
Array out-of-bounds	`for(i=0; i<=N; ++i) arr[i]`	Loop returns early	Loop optimized away
Infinite loop (no side effects)	`while(true) { if(found) return; }`	Must terminate eventually	Loop removed
Use-after-free	`delete p; *p = 5;`	Freed memory not accessed	Unpredictable

📌 5. Detection Strategy

Recommended Toolchain:

Tool	Flag	Detects	When to Use
AddressSanitizer	`-fsanitize=address`	Null deref, bounds, use-after-free	Every dev build
UBSanitizer	`-fsanitize=undefined`	Integer overflow, null ptr	Every dev build
ThreadSanitizer	`-fsanitize=thread`	Data races	Multithreaded testing
Compiler warnings	`-Wall -Wextra -Werror`	Common mistakes	Always
Static analysis	Clang-Tidy, Coverity	Complex patterns	CI/CD pipeline

Development Command:

```
g++ -std=c++20 -O2 -g \
    -Wall -Wextra -Wpedantic -Werror \
    -fsanitize=address,undefined \
    -fno-omit-frame-pointer \
    program.cpp
```

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Edge Cases

📌 1. The Fermat's Last Theorem "Disproof" (1/3)

```
#include <iostream>

// This function attempts to find a counterexample to Fermat's Last Theorem
// a³ + b³ = c³ (which has no integer solutions)
int fermat_disproof() {
    const int MAX = 1000;
    int a = 1, b = 1, c = 1;

    // Search for counterexample
    while (true) {  // ❌ Infinite loop with no side effects = UB
        if (a*a*a == b*b*b + c*c*c) {
            std::cout << "Fermat's Last Theorem disproved!\n";
            std::cout << a << "³ + " << b << "³ = " << c << "³\n";
            return 1;
        }

        ++a;
        if (a > MAX) { a = 1; ++b; }
        if (b > MAX) { b = 1; ++c; }
        if (c > MAX) { c = 1; }  // Loop resets, making it truly infinite
    }

    return 0;  // Never reached
}

// Compiled with -O3 (aggressive optimization)
// OUTPUT: Returns 0 immediately, loop is REMOVED entirely!
```

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Edge Cases

📌 1. The Fermat's Last Theorem "Disproof" (2/3)

What Happens:

With optimization enabled, the compiler recognizes:

1. The loop is infinite (no break statement that can be reached in reasonable time)
2. The loop has no observable side effects (the `if` with `cout` is never true)
3. Infinite loops with no side effects are **undefined behavior** (C++11 §1.10/24)
4. Since UB never occurs in correct programs, the loop can be **removed entirely**
5. The function optimizes to `return 0;`

Why It's Dangerous:

- The code appears to work correctly (it's searching for something)
- It passes code review (looks like valid search logic)
- Debug builds work as expected (no optimization)
- Release builds mysteriously "skip" the expensive computation

Real-World Impact:

This pattern appears in search algorithms, verification loops, and timeout implementations that can be unexpectedly optimized away.

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Edge Cases

📌 1. The Fermat's Last Theorem "Disproof" (3/3)

Autonomous Vehicle Context:

```
bool wait_for_sensor_calibration(Sensor& sensor) {
    const int MAX_ATTEMPTS = 1000;
    int attempts = 0;

    while (true) {
        if (sensor.is_calibrated()) {
            return true;
        }

        ++attempts;
        if (attempts > MAX_ATTEMPTS) {
            break;  // ✅ This break makes it NOT infinite
        }

        std::this_thread::sleep_for(std::chrono::milliseconds(10));  // ✅ Side effect
    }

    return false;
}
```

Fix: Add observable side effects (I/O, thread sleep, volatile access) or guaranteed termination.

📌 2. Null Pointer Check Removal After Dereference (1/3)

```
#include <iostream>

// ❌ DANGEROUS: Appears to have safety check, but optimizer removes it
void process_sensor_data(int* data) {
    // Dereference happens first
    int value = *data;  // If data is null, this is UB

    // ... 50 lines of complex processing ...
    int processed = value * 2 + 10;
    int result = processed / (value - 5);

    // Safety check (or so it appears)
    if (data == nullptr) {  // ❌ Compiler removes this check!
        std::cerr << "ERROR: Null pointer detected!\n";
        return;
    }

    // Use the data
    std::cout << "Processed value: " << result << "\n";
}

// With optimization, the null check is COMPLETELY REMOVED
// Compiler reasoning:
// "If we reached the null check, the dereference *data must have succeeded,
//  therefore data cannot be null, so the check always fails,
//  so I can remove the entire if block."
```

What Happens:

1. `*data` dereferences the pointer
2. If `data` is null, that's UB—program can do anything
3. Compiler assumes UB never happens (because your code is "correct")
4. Therefore, if we reach the null check, `data` must be non-null
5. The null check is **provably false** and gets removed

📌 2. Null Pointer Check Removal After Dereference (2/3)

Disassembly Comparison:

```
; Without optimization (-O0)
mov eax, [rdi]        ; *data
; ... processing ...
test rdi, rdi         ; if (data == nullptr)
je .error_label       ; jump if zero

; With optimization (-O3)
mov eax, [rdi]        ; *data
; ... processing ...
; (null check completely absent)
```

Real CVE Example:

LibTIFF CVE-2019-14973: Similar pattern where overflow checks were optimized away after operations that would trigger UB if overflow occurred.

📌 2. Null Pointer Check Removal After Dereference (3/3)

Autonomous Vehicle Context:

```
// ❌ WRONG: Check after use
void update_vehicle_state(SensorReading* reading) {
    current_velocity = reading->velocity;  // Dereference first

    if (reading == nullptr) {  // Removed by optimizer!
        log_error("Null sensor reading");
        return;
    }

    apply_velocity(current_velocity);
}

// ✅ CORRECT: Check before use
void update_vehicle_state(SensorReading* reading) {
    if (reading == nullptr) {  // Check FIRST
        log_error("Null sensor reading");
        return;
    }

    current_velocity = reading->velocity;  // Dereference after check
    apply_velocity(current_velocity);
}
```

Key Lesson: Always validate pointers **before** dereferencing, never after.

📌 3. Integer Overflow Check Removal (1/3)

```
#include <iostream>

// Attempt to detect signed integer overflow
bool will_overflow_addition(int x, int y) {
    // ❌ This check is REMOVED by the optimizer!
    if (x + y < x) {  // Detect overflow by wraparound
        std::cout << "Overflow detected!\n";
        return true;
    }
    return false;
}

// Compiled with -O2:
// The function ALWAYS returns false!
// Compiler reasoning:
// "Signed overflow is UB (C++ standard §5/4)
//  I can assume it never happens
//  Therefore x + y >= x is always true
//  Therefore x + y < x is always false
//  Remove the entire if block"

// ✅ CORRECT: Use unsigned arithmetic or check before operation
bool will_overflow_addition_correct(int x, int y) {
    // Method 1: Check before operation
    if (y > 0 && x > INT_MAX - y) {
        return true;
    }
    if (y < 0 && x < INT_MIN - y) {
        return true;
    }
    return false;
}

// Method 2: Use unsigned arithmetic (wrapping is defined)
bool will_overflow_unsigned(unsigned int x, unsigned int y) {
    unsigned int sum = x + y;
    return sum < x;  // ✅ Works for unsigned (wrapping is defined)
}
```

📌 3. Integer Overflow Check Removal (2/3)

Why Signed Overflow Detection Fails:

- Signed integer overflow is **undefined behavior** (C++11 §5/4)
- The compiler assumes UB never occurs
- Therefore `x + y` cannot overflow
- Therefore `x + y < x` is always false
- The check is optimized away

Contrast with Unsigned:

```
unsigned int x = UINT_MAX;
unsigned int y = 1;
unsigned int sum = x + y;  // ✅ Defined behavior: wraps to 0
// sum < x is true, overflow detected
```

Real-World Vulnerabilities:

- **LibTIFF CVE-2019-14973**: Multiple overflow checks removed
- **Google Native Client**: Security vulnerability from undefined shift operation

Autonomous Vehicle Context:

```
// ❌ DANGEROUS: Overflow check removed
bool safe_to_accelerate(int current_speed, int acceleration) {
    int new_speed = current_speed + acceleration;
    if (new_speed < current_speed) {  // Removed!
        return false;  // Overflow detected
    }
    return new_speed <= MAX_SAFE_SPEED;
}

// ✅ SAFE: Check before operation
bool safe_to_accelerate(int current_speed, int acceleration) {
    if (acceleration > MAX_SAFE_SPEED - current_speed) {
        return false;  // Would overflow
    }
    return true;
}
```

📌 3. Integer Overflow Check Removal (3/3)

Detection Tools:

```
# Compile with sanitizers to detect overflow at runtime
g++ -fsanitize=signed-integer-overflow -g program.cpp

# Use compiler warnings
g++ -Wall -Wextra -Wsign-conversion program.cpp
```

📌 4. Array Bounds Check Optimization (1/3)

```
#include <iostream>

// Search for value in 4-element array
bool exists_in_array(int* arr, int needle) {
    for (int i = 0; i <= 4; ++i) {  // ❌ BUG: should be i < 4
        if (arr[i] == needle) {
            return true;
        }
    }
    return false;
}

// With optimization, this becomes:
// bool exists_in_array(int* arr, int needle) {
//     return true;  // ❌ ALWAYS returns true!
// }
```

Compiler's Reasoning:

1. When `i == 4`, accessing `arr[4]` is out of bounds (UB)
2. If the function returns false, all loop iterations completed
3. But completing iteration with `i == 4` causes UB
4. UB never happens in correct programs
5. Therefore, the loop must have returned true before reaching `i == 4`
6. The function can be optimized to `return true;`

📌 4. Array Bounds Check Optimization (2/3)

Proof via Compiler Explorer:

```
// Input code
bool exists_in_array(int* arr, int needle) {
    for (int i = 0; i <= 4; ++i) {
        if (arr[i] == needle) return true;
    }
    return false;
}

// GCC -O3 output (x86-64)
exists_in_array(int*, int):
    mov eax, 1        ; return true
    ret               ; that's it - no loop!
```

📌 4. Array Bounds Check Optimization (3/3)

Autonomous Vehicle Context:

```
// Sensor array check
const int NUM_SENSORS = 4;

// ❌ DANGEROUS: Loop bound off by one
bool is_sensor_active(Sensor* sensors, int sensor_id) {
    for (int i = 0; i <= NUM_SENSORS; ++i) {  // BUG!
        if (sensors[i].id == sensor_id) {
            return sensors[i].is_active;
        }
    }
    return false;
}

// With optimization: Always returns false (or random data from out-of-bounds)
// Even worse: May access sensors[4] which is UB

// ✅ CORRECT: Proper loop bounds
bool is_sensor_active(Sensor* sensors, int sensor_id) {
    for (int i = 0; i < NUM_SENSORS; ++i) {  // ✅ Correct
        if (sensors[i].id == sensor_id) {
            return sensors[i].is_active;
        }
    }
    return false;
}
```

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Code Examples

📌 1. Detecting Optimizer Behavior with Godbolt

How to Check:

1. Go to https://godbolt.org
2. Paste code
3. Compare assembly for `-O0` vs `-O3`
4. First version: Check disappears with optimization
5. Second version: Check remains (no UB)

```
#include <iostream>

// Example showing how optimizer removes checks

// ❌ DANGEROUS: Overflow check
int add_with_check(int a, int b) {
    int sum = a + b;
    if (sum < a) {  // Overflow check - REMOVED by optimizer
        std::cout << "Overflow detected!\n";
        return INT_MAX;
    }
    return sum;
}

// ✅ SAFE: Check before operation
int add_with_check_correct(int a, int b) {
    if (b > 0 && a > INT_MAX - b) {  // Check before overflow
        std::cout << "Overflow would occur!\n";
        return INT_MAX;
    }
    if (b < 0 && a < INT_MIN - b) {
        std::cout << "Underflow would occur!\n";
        return INT_MIN;
    }
    return a + b;
}

// View on Compiler Explorer (godbolt.org):
// g++ -O0 (no optimization): Both versions have checks
// g++ -O3 (optimization): First version's check is REMOVED
```

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Code Examples

📌 2. Move Semantics in Practice (1/2)

```
#include <iostream>
#include <vector>
#include <string>
#include <chrono>

class LargeSensorData {
public:
    std::vector<double> readings;  // 1 million readings
    std::string metadata;

    LargeSensorData(size_t size) : readings(size, 0.0), metadata("Sensor Data") {
        std::cout << "Constructed with " << size << " readings\n";
    }

    // ❌ WRONG: Doesn't actually move
    LargeSensorData(LargeSensorData&& other)
        : readings(other.readings),  // COPIES the vector!
          metadata(other.metadata)   // COPIES the string!
    {
        std::cout << "Move constructor (WRONG version)\n";
    }

    // ✅ CORRECT: Actually moves
    LargeSensorData(LargeSensorData&& other) noexcept
        : readings(std::move(other.readings)),  // Moves the vector
          metadata(std::move(other.metadata))   // Moves the string
    {
        std::cout << "Move constructor (CORRECT version)\n";
    }

    LargeSensorData(const LargeSensorData&) = delete;  // Prevent accidental copies
    LargeSensorData& operator=(const LargeSensorData&) = delete;
};

void benchmark_move_semantics() {
    using namespace std::chrono;

    const size_t DATA_SIZE = 1'000'000;
    const int ITERATIONS = 1000;
// ...continues on next slide
```

📌 2. Move Semantics in Practice (2/2)

[Code continues across slides]

```
// ...continued from previous slide

    auto start = high_resolution_clock::now();

    for (int i = 0; i < ITERATIONS; ++i) {
        LargeSensorData data(DATA_SIZE);
        LargeSensorData moved = std::move(data);  // Move operation
    }

    auto end = high_resolution_clock::now();
    auto duration = duration_cast<milliseconds>(end - start);

    std::cout << "Total time: " << duration.count() << "ms\n";
    std::cout << "Average per move: " << duration.count() / (double)ITERATIONS << "ms\n";
}

// Performance difference:
// Wrong version (copy): ~1000ms (memcpy 1M doubles * 1000 times)
// Correct version (move): ~10ms (pointer swap * 1000 times)
// 100x speedup!
```

Advanced C++ Pitfalls - Compiler Optimizations, UB, and Modern C++ Traps

Code Examples

📌 3. Perfect Forwarding Demonstration (1/2)

```
#include <iostream>
#include <utility>
#include <vector>

void process(const std::vector<int>& v) {
    std::cout << "Lvalue overload: " << v.size() << " elements\n";
}

void process(std::vector<int>&& v) {
    std::cout << "Rvalue overload: " << v.size() << " elements (moving)\n";
    v.clear();  // Consume the rvalue
}

// ❌ Imperfect forwarding
template<typename T>
void bad_forward(T arg) {
    process(arg);  // Always calls lvalue overload (arg is lvalue)
}

// ✅ Perfect forwarding
template<typename T>
void good_forward(T&& arg) {
    process(std::forward<T>(arg));  // Preserves value category
}

int main() {
    std::vector<int> vec = {1, 2, 3};

    std::cout << "=== Bad Forward ===\n";
    bad_forward(vec);                        // Lvalue
    bad_forward(std::vector<int>{4, 5, 6});  // Rvalue, but still calls lvalue overload!

    std::cout << "\n=== Good Forward ===\n";
    good_forward(vec);                        // Lvalue → calls lvalue overload
    good_forward(std::vector<int>{7, 8, 9}); // Rvalue → calls rvalue overload
}

// Output:
// === Bad Forward ===
// ...continues on next slide
```

📌 3. Perfect Forwarding Demonstration (2/2)

[Code continues across slides]

```
// ...continued from previous slide
// Lvalue overload: 3 elements
// Lvalue overload: 3 elements  ← WRONG! Should be rvalue
//
// === Good Forward ===
// Lvalue overload: 3 elements
// Rvalue overload: 3 elements (moving)  ← CORRECT!
```

What You Learned

Key Takeaways

- ✓ The Optimizer's Fundamental Assumption
- ✓ Categories of Advanced Pitfalls
- ✓ Why This Is Dangerous
- ✓ Undefined Behavior Categories

→ *Swipe to learn* →